

SALESTORM

Security & Observability Design

Team: Binary Minds Members: Jumaanah Basheeth , Harini C, Manasa Devi Date: 5 October 2026

1. Purpose and Scope

This document covers requirement NFR-14 (security) and NFR-15 (observability). The flash sale concentrates money, scarce stock and hostile traffic (bots, scalpers, floods) into a few minutes, so security controls must work at very high request rates, and observability must let the team prove that no unit was oversold and no paid customer was lost.

2. Security Design

2.1 Threat summary

Threat	Flash-sale example	Main controls
Bots and scalpers	Scripts buy the limited stock before real customers	WAF bot rules, per-user rate limits, one unit per customer, waiting room, challenge (CAPTCHA) for risky traffic
DDoS / traffic flood	Traffic far above plan, or deliberate flooding	CDN and WAF absorb at the edge, load shedding at the gateway, cached sold-out response
Credential stuffing / account takeover	Stolen passwords used to buy	Identity provider with MFA option, login throttling, anomaly detection on new devices
Replay and double submit	Same request sent twice, or a captured request replayed	Idempotency keys, short-lived tokens, TLS, webhook timestamp window
Broken access control (IDOR)	Customer reads or cancels someone else's order by changing an id	Ownership check in every service, not only at the gateway
Tampered input	Client sends a lower price or quantity 100	Server-side pricing, schema validation, quantity limited to 1
Forged payment webhook	Attacker marks an unpaid order as paid	HMAC signature, timestamp window, IP allow-list, and a status lookup with the gateway before trusting an unusual result
Injection	SQL or script injection in fields	Parameterised queries only, input validation, output encoding
Card data exposure	Card number stored or logged	Hosted payment fields and tokens only, no card data in SALESTORM systems or logs
Leaked secrets / insider misuse	Key in code, or admin changes stock silently	Vault, rotation, least privilege, audit log of every admin action

2.2 Authentication and authorization

- **Authentication:** OAuth2/OIDC through an identity provider. The API Gateway validates the JWT (signature, expiry, audience). Access tokens are short-lived (example 15 minutes) with refresh tokens.
- **Authorization:** role-based plus ownership checks. The gateway checks the role. Each service re-checks that the resource belongs to the caller, so a gateway bypass still cannot expose data.
- **Service-to-service:** mutual TLS with a workload identity per service. A service can call only the services it needs (for example Checkout can call Inventory and Payment, but Notification cannot call Payment).
- **Data stores:** each service has its own database account with least privilege (Inventory cannot read Payment tables).

Role	Can do	Cannot do
Customer	Browse, reserve, pay, view and cancel own orders	See other customers' data, any admin route
Support	Read orders and payments, view tracking	Change stock, trigger refunds without approval, replay messages
Admin (Ops)	View metrics, trigger reconciliation, rebuild stock gate, replay dead letters. Every action is audited	Edit payment records directly or delete audit rows
Service identity	Only the internal calls listed in the API spec	Any public or admin route

2.3 Transport and API security

- HTTPS everywhere (TLS 1.2 minimum, 1.3 preferred), HSTS, and certificates managed and rotated automatically.
- Internal traffic uses mTLS. Databases, Redis and the broker are in private subnets with no public access.
- The gateway validates requests against the OpenAPI schema, enforces body-size limits, and sets security headers and strict CORS.
- Errors use the standard problem format and never leak stack traces or internal ids.

2.4 Rate limiting and abuse protection

Layer	Control	Example limit
CDN / WAF	Bot management, IP reputation, DDoS protection, geo rules	Managed rules
Gateway: per IP	Token bucket in Redis. Generous, because many users share one NAT address	300 requests/min/IP
Gateway: per user	Strict on the sensitive routes	POST /reservations 5/min, POST /checkout 5/min, status polling 60/min
Waiting room	Admits users at a rate the backend can serve, so overload becomes a short wait instead of errors	Set from load-test capacity
Business rule	One unit per customer per sale, so extra accounts and retries gain nothing	PER_CUSTOMER_LIMIT
Risk challenge	CAPTCHA or proof-of-work when a user or IP looks automated	Triggered by score
Idempotency	Retries and replays cannot create a second effect	24 hour key retention

If the rate limiter itself fails: the reservation and checkout routes **fail closed** (a small in-memory limit per gateway pod takes over), because protecting inventory matters more than convenience. Read-only routes **fail open**.

2.5 Input validation

- Validate at the gateway (schema) and again in each service (never trust the caller). Use allow-lists for enums, types, lengths and ranges.
- Quantity must be 1. Price, discount and total are always **calculated on the server** from sale and coupon data. Any client-sent price is ignored.
- Only parameterised queries are used. No string-built SQL.
- Free-text fields are length-limited and encoded on output. Uploaded files are not accepted.

2.6 Payment data handling

- **Reduced PCI scope:** card details are entered into the gateway's hosted fields and exchanged for a token. SALESTORM stores and sees only the token, a gateway transaction id and (optionally) the last four digits. No card number or CVV is ever stored or logged.

- The gateway is called over TLS with credentials from the vault. Each charge carries our `transaction_ref` as the idempotency key.
- Webhooks are accepted only with a valid HMAC signature, a timestamp within 5 minutes and a known source IP, and are de-duplicated by event id.
- The Payment database is separate, encrypted at rest and reachable only by the Payment Service.

2.7 Data protection

- Encryption at rest (AES-256 through the cloud KMS) for databases, backups and the message broker. Field-level encryption for sensitive personal fields such as phone number and address.
- Personal data is masked in logs and never used as a metric label. Retention periods are defined, and personal data is deleted or anonymised after the retention period, in line with applicable data-protection law.

2.8 Audit logging

- **What is audited:** every reservation, payment and order state change, every admin action (rebuild stock gate, reconcile, replay dead letter), and security events (failed logins, signature failures, rate-limit blocks).
- **Fields:** timestamp, actor (customer, service or admin), action, entity type and id, old and new status, source IP, trace id. This matches the `AUDIT_LOG` table in the ER diagram.
- **Protection:** audit records are append-only and shipped to write-once storage separate from application logs. Nobody, including admins, can edit or delete them. They are kept for the required retention period.
- Audit writes for state changes happen in the same transaction as the change, so an action cannot occur without its record.

2.9 Secrets management

- All secrets (database passwords, gateway keys, webhook secret, signing keys) live in a vault with KMS-backed encryption and are injected at runtime. Nothing is stored in code, images or plain environment files.
- Rotation: automatic (example every 90 days), with an overlap window for gateway and webhook keys so rotation causes no downtime. Database credentials are short-lived where the vault supports it.
- Secret scanning runs in CI and blocks commits that contain keys.

2.10 Security trade-offs

Choice	What it costs
Strict per-user limits and challenges	Some real users are blocked or annoyed (false positives). Limits need tuning from real traffic.
Waiting room	Adds delay and a more complex user flow, but prevents overload failures.
mTLS and per-service identities	More operational work (certificates, policies) in exchange for a smaller blast radius.
Fail-closed limiter on reserve	During a Redis outage some legitimate requests are rejected, which is accepted to protect stock.
Tokenised payments	Dependence on the gateway's hosted fields, but SALESTORM stays out of most card-data compliance scope.

3. Observability Design

3.1 Approach

Three signals, all correlated by one trace id: **metrics** (is something wrong?), **logs** (what happened?) and **traces** (where did this request go?). Services emit telemetry with OpenTelemetry. Example stack: Prometheus and Grafana for metrics, a central log store (Loki or ELK), Jaeger or Tempo for traces, and Alertmanager for paging.

3.2 Key metrics

Metric	Type	Labels	What it tells us
http_requests_total	Counter	service, route, status	Request rate and error rate by route
http_request_duration_seconds	Histogram	service, route	p50, p95, p99 latency (target: reserve p95 < 300 ms)
reservation_attempts_total	Counter	result (created, out_of_stock, duplicate, limit, error)	Inventory reservation outcomes and failures
stock_gate_rejections_total	Counter	gate (redis, db)	How much load the Redis gate absorbs
reservations_active	Gauge		Reservations waiting for payment
reservations_expired_total	Counter	reason	Released by timeout or payment failure
inventory_available / reserved / sold	Gauge	product	Live stock counts (from the DB)
inventory_invariant_violation	Gauge	product	1 if available + reserved + sold differs from total stock. Must stay 0
redis_db_stock_drift	Gauge	product	Difference between the Redis counter and the DB value
payment_attempts_total	Counter	result (succeeded, failed, unknown)	Payment failure rate (expected about 5%)
payment_gateway_latency_seconds	Histogram	provider	Gateway health and timeout risk
circuit_breaker_state	Gauge	provider	Closed, open or half-open
orders_created_total	Counter	status	Orders confirmed
order_conversion_ratio	Gauge	sale	Confirmed orders divided by successful reservations
paid_without_order_count	Gauge		Payments succeeded but no order after 2 minutes. Must return to 0
outbox_pending_count / oldest_age_seconds	Gauge	service	Events waiting to be published
consumer_lag	Gauge	topic, group	Queue backlog per consumer
dlq_depth	Gauge	topic	Messages that failed repeatedly
reconciliation_findings_total	Counter	type	Problems found and repaired by the safety-net job

Cardinality rule: labels are limited to small fixed sets. Customer ids, reservation ids and order ids are never metric labels. They belong in logs and traces.

3.3 Business funnel

Track one funnel per sale: **requests, then reservation attempts, then successful reservations, then payment attempts, then payments succeeded, then orders confirmed**. The ratio of orders confirmed to successful reservations is the order conversion rate. In the test case the expected value is about 95% (5%

payment failures), and the released 5% of stock is resold. A sudden fall in conversion points to a payment or order-path problem.

3.4 Structured logs

Logs are JSON, one event per line, with the same base fields everywhere:

```
{ "ts": "2026-10-05T10:12:31.482Z", "level": "INFO", "service": "inventory",  
  "event": "reservation.created", "traceId": "4bf92f35...", "requestId": "r-81f2",  
  "reservationId": "b1d5c8a2-...", "productId": 9001, "customerRef": "c_7f31a9",  
  "outcome": "RESERVED", "durationMs": 14 }
```

Critical business event	Emitted by	Extra fields
reservation.created / rejected / duplicate	Inventory	reservationId, reason (out_of_stock, limit, replay), gate result
reservation.confirmed / released / expired	Inventory	reservationId, old and new status, reason
payment.initiated / succeeded / failed / unknown	Payment	paymentId, transactionRef, provider, gateway latency
payment.reconciled / refunded	Payment, Reconciliation	paymentId, outcome, refund reason
order.created / confirmed / cancelled	Order	orderId, reservationId, status
event.published / consumed / duplicate_ignored / dead_lettered	All services	eventType, eventId, attempt, consumer
security.rate_limited / signature_invalid / auth_failed	Gateway, Payment	route, source IP (hashed where required), rule
admin.action	Admin APIs	actor, action, target

- customerRef is a pseudonymous id, not an email or phone. Card data, tokens and secrets are never logged.
- All business events are always logged. Debug-level logs are sampled or switched on temporarily for a single trace id.

3.5 Distributed tracing

- W3C traceparent is created at the gateway and passed on every HTTP call.
- The trace context is stored in the **outbox row** and sent as a Kafka header, so one trace continues from checkout through payment, the broker, order creation and shipment, even though those steps are asynchronous.
- Sampling at flash-sale volume:** tracing 500,000 requests per second is not affordable. Use about 1% head sampling for normal traffic, **plus 100% of successful reservations, all checkouts, all errors and all slow requests** (tail sampling). Only about 100 reservations succeed, so every purchase that matters is fully traced.

Span	Service	What to look at
POST /reservations	Gateway, Checkout	Total latency, rate-limit decision
stock gate (Redis Lua)	Inventory	Gate result and time
reserve transaction	Inventory, DB	Rows updated, lock wait, commit time
POST /checkout, charge	Checkout, Payment, Gateway	Gateway latency, retries, circuit state
outbox publish, consume	Payment, Broker, Order	Queue delay between payment and order

Span	Service	What to look at
create order, publish OrderConfirmed	Order, DB	Idempotency hit or new order
create shipment, notify	Shipment, Notification	Downstream delays (non-critical)

3.6 Alerts

Alert	Condition (example)	Severity	First action
Inventory invariant broken	inventory_invariant_violation = 1, or sold quantity above total stock	Critical, page	Pause new reservations for the product, compare DB with audit log, start reconciliation
Redis and DB drift	redis_db_stock_drift is not 0 for 1 minute	High	Rebuild the gate from the DB (admin endpoint)
Payment failure spike	Failed payments above 10% for 5 minutes (normal about 5%)	High (page above 25%)	Check gateway status and circuit breaker, review decline codes
Payments stuck unknown	Any payment UNKNOWN longer than 5 minutes	High	Run reconciliation, query gateway by transaction_ref
Circuit breaker open	Payment circuit open for more than 1 minute	High	Confirm gateway outage, show a retry message, keep reservations valid
Paid without order	paid_without_order_count above 0 for 2 minutes	Critical, page	Check Order Service and consumer lag, replay or run reconciliation
Queue backlog	Consumer lag above 10,000 messages, or oldest message older than 60 seconds	High	Scale consumers, check for a poison message
Dead-letter messages	dlq_depth above 0 on payment or order topics	High	Inspect and replay after the fix
Outbox stalled	Oldest pending outbox event older than 30 seconds	High	Check the outbox relay and broker health
Reserve latency high	p99 above 800 ms for 5 minutes	Medium	Check gate, DB lock waits and autoscaling
Error rate high	5xx above 1% for 5 minutes	High	Check recent deploys and dependency health
Database trouble	Replication lag above 5 seconds, or failover event	High	Confirm failover, verify no lost writes
Security anomaly	Sharp rise in rate-limit blocks or signature failures	Medium	Review the WAF, tighten rules if an attack is in progress

To limit alert fatigue, alerts are tied to customer impact or guarantee risk. Low-level signals (CPU, memory) are dashboards, not pages.

3.7 Dashboards and SLOs

- **Sale war-room dashboard:** stock (available, reserved, sold), requests per second, reserve p95 and p99, funnel and conversion, payment success rate, circuit state, queue lag, DLQ depth, paid-without-order count.
- **Per-service dashboards:** the usual rate, errors and duration, plus dependency health.

SLO (example)	Target
Reserve path availability	99.9% of requests answered without a server error
Reserve latency	p95 under 300 ms and p99 under 800 ms
Sold-out response	Under 100 ms without touching the database
Order creation after payment	99% within 60 seconds, 100% eventually (guarantee)

SLO (example)	Target
Reservation release after expiry	Within 30 seconds

3.8 First-response runbook (top incidents)

Incident	First steps
Inventory mismatch	Stop new reservations for the product. Compare DB counts with reservation and audit records. Run reconciliation. Rebuild the Redis counter from the DB.
Payment gateway failing	Confirm circuit state. Keep reservations valid. Show "payment temporarily unavailable, retry". After recovery, reconcile all UNKNOWN payments by transaction_ref.
Order Service down	Do nothing destructive. Events wait in the queue. Watch lag and paid-without-order count. After recovery confirm lag drains and run reconciliation.
Redis down	The DB-only gate takes over. Watch DB latency, tighten rate limits, rebuild the counter after Redis returns.

3.9 Observability trade-offs

- Telemetry volume at very high traffic is costly, so we use sampling, with full coverage for the rare events that matter (successful reservations, payments, errors).
- High-cardinality labels would overload the metrics system, so identifiers go to logs and traces instead.
- More alerts mean faster detection but more noise, so each alert is tied to a guarantee or a customer impact.